Practical Multicriteria Urban Bicycle Routing

Jan Hrnčíř, Pavol Žilecký, Qing Song, and Michal Jakob

***Abstract*—Increasing the adoption of cycling is crucial for achieving more sustainable urban mobility. Navigating larger cities on a bike is, however, often challenging due to the cities' fragmented cycling infrastructure and/or complex terrain topology. Cyclists would thus benefit from intelligent route planning that would help them discover routes that best suit their transport needs and preferences. Because of the many factors cyclists consider in deciding their routes, employing a multicriteria route search is vital for properly accounting for cyclists' route-choice criteria. A direct application of optimal multicriteria route search algorithms is, however, not feasible due to their prohibitive computational complexity. In this paper, we formalize a multicriteria bicycle routing problem and propose several heuristics for speeding up the multicriteria route search. We evaluate our method on a real-world cycleway network and show that speedups of up to four orders of magnitude over the standard multicriteria label-setting algorithm are possible with a reasonable loss of solution quality. Our results make it possible to practically deploy bicycle route planners capable of producing diverse high-quality route suggestions respecting multiple real-world route-choice criteria.**

***Index Terms*—Bicycle routing, multicriteria shortest path, heuristic speedups.**

I. Introduction

UTILITY cycling, i.e., using the bicycle as a mode of transport, is the original and the most common type of cycling in the world [1]. Cycling provides a convenient and affordable form of transport for most segments of the population. It has a range of health, environmental, economical, and societal benefits and is therefore promoted as a modern, sustainable mode of transport [2], [3].

In contrast to car drivers, cyclists consider a significantly broader range of factors while deciding their routes. By employing questionnaires and GPS tracking, researchers have found that besides travel time and distance, cyclists are sensitive to slope, turn frequency, junction control, noise, pollution, scenery, and traffic volumes [4], [5]. Moreover, the relative importance of these factors varies among cyclists and can also be affected by weather and the purpose of the trip [4].

Finding routes that properly take all the above factors into account is no easy task, particularly when cycling in complex urban environments. Consequently, cyclists would benefit from intelligent route planning software to help them discover routes that best suit their transport needs and preferences. Such route planners would be particularly useful for inexperienced cyclists with limited knowledge of their surroundings but they would also benefit experienced riders who want to fine-tune their routes [6], in effect making cycling a more attractive and accessible transport option. The same also holds for travelers in multimodal public transport networks [7] where the software can guide the user on his or her multimodal trip.

The vast majority of existing approaches to bicycle routing, however, do not use multi-criteria search methods and they thus cannot produce diverse sets of suggested routes properly accounting for cyclists' multiple route-choice criteria.

We build on our previous work [8] where an optimal tri-criteria shortest path algorithm for multi-criteria bicycle routing has been applied. Unfortunately, the proposed algorithm is slow on realistic problem instances and cannot be used for interactive route planning. In this paper, we overcome the above limitations and present the first bicycle routing algorithm that properly considers multiple realistic cyclists' route-choice criteria yet is fast enough for interactive use. Our algorithm extends the well-known multi-criteria label-setting algorithm [9] with several speedup heuristics in order to generate, in a much shorter time, routes that closely approximate the full set of Pareto optimal routes. In contrast to the majority of existing work, our algorithm employs a formulation of the multi-criteria bicycle routing problem that incorporates realistic route choice factors based on recent studies of cyclists' behavior [4], [5]. We thoroughly evaluate our algorithm in terms of the speed and quality of suggested routes on a diverse set of real-world urban areas.

The paper is organized as follows. In Section II, we explain how our approach fits within the existing work on routing and multi-criteria shortest path search. In Section III, we present our first contribution—a formal model of the multi-criteria bicycle routing problem together with the process through which instances of multi-criteria bicycle routing problems can be instantiated from real-world data. In Section IV, we present our second and main contribution—first describing the novel heuristic-enabled multi-criteria Dijkstra algorithm and then the specific speedup heuristics proposed. In Section V, we

Manuscript received August 7, 2015; revised February 17, 2016 and May 12, 2016; accepted May 28, 2016. Date of publication July 22, 2016; date of current version February 24, 2017. This work was supported in part by the European Social Fund within the framework of realizing the project "Support of intersectoral mobility and quality enhancement of research teams at Czech Technical University in Prague" under Grant CZ.1.07/2.3.00/30.0034 (period of the project's realization is December 1, 2012–June 30, 2015); by the European Commission under MyWay, a collaborative project part of the Seventh Framework Programme for research, technological development, and demonstration under Grant 609023; by the Czech Technical University under Grant SGS13/210/OHK3/3T/13; and by the program "Projects of Large Infrastructure for Research, Development, and Innovations" under Grant LM2010005. The Associate Editor for this paper was H. Jula.

The authors are with the Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague, 12135 Prague, Czech Republic (e-mail: hrncir@agents.fel.cvut.cz; zilecky@agents.fel.cvut.cz; song@agents.fel.cvut.cz; jakob@agents.fel.cvut.cz).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TITS.2016.2577047

present an extensive experimental evaluation of our approach on realistic cycleway networks. Importantly, this evaluation provides detailed insights into the relative performance of various speedup heuristics and their combinations, and it should be therefore viewed as a valuable contribution on its own. Finally, Section VI concludes.

II. Related Work

Multi-criteria shortest path problem is studied in the literature for a long time. As far as general multi-criteria shortest path algorithms are concerned, the multi-criteria label-setting (MLS) algorithm [9], [10] extends Dijkstra's algorithm [11] by operating on labels that have multiple cost values. For each node, the algorithm stores a bag of non-dominated labels. The priority queue stores labels (typically in a lexicographic order) instead of nodes as in Dijkstra's algorithm. A minimum label from the priority queue is processed in every iteration. On the contrary, the multi-criteria label-correcting (MLC) algorithm [12], [13] processes the whole bag of nondominated labels associated with a current node at once.

The main parameter that affects the runtime of the algorithm is the size of the Pareto set. In general, the Pareto set can be exponentially large in the input graph size [14]. That leads to runtimes that are not practically usable. Recently, heuristic accelerations have attracted considerable attention, aiming at finding a set of routes that is similar to the optimal Pareto solution. First, the search space can be pruned using the ellipse around the origin and destination [15]. Second, in [16], the authors proposed a near-admissible multi-criteria search algorithm to approximate the optimal set of Pareto routes in a state space graph by using the ϵ-dominance approach. It has been proven that the $(1+\epsilon)$-Pareto sets have a polynomial size [17]. Third, in [18], the authors developed several heuristics to weaken the domination rules during the search (e.g., using buckets for the criteria values). Fourth, in [19], the authors proposed a modified label-correcting algorithm with a new label-selection strategy and dominance conditions for a three-criteria unmanned combat vehicle routing problem.

An alternative approach is represented by MOA* [20] and NAMOA* [21], which are multi-criteria extensions of the standard A* algorithm [22]. The extension lies in using a vector of heuristics and a search graph, i.e., a directed acyclic graph to record the set of non-dominated paths to visited nodes. Every time a new path reaches a node, its cost vector is checked for dominance with the set of all stored path cost vectors at that node. The NAMOA* algorithm with the Tung & Chew heuristic [23] was recently shown to achieve an order of magnitude speedup for bicriteria road routing [24].

In addition to general multi-criteria graph search, multi-criteria journey and route planning methods designed specifically for searching graphs representing transport networks have already been widely studied. The solutions range from multi-criteria route planning in train networks [25], [26] to multi-modal route planning using a combination of public transport and individual transport modes (e.g., car, taxi, shared bike) [18], [27]. The solutions usually optimize for travel time, number of transfers, and walking distance criteria.

In contrast to car and public transport route planning for which advanced algorithms and mature software implementations exist, *bicycle route planning* is a relatively underexplored topic. So far, only basic algorithms have been used for bicycle routing. In particular, despite the highly multi-criteria nature of cyclists' route-choice preferences, almost all existing approaches to bicycle routing do not use multi-criteria search methods to properly account for such a multi-criteriality. Instead, these existing bicycle routing approaches transform multi-criteria search to single-criterion search either by optimizing each criteria separately [28], [29] or by using a weighted combination of all criteria [30], [31].

Unfortunately, the scalarization of multi-criteria problems using a linear combination of criteria may miss many Pareto optimal routes [32], [33] and consequently reduce the quality and relevance of suggested routes. Scalarization also requires the user to weight the importance of individual route criteria *a priori*, which is difficult for most users.

Only in the last few years, multi-criteria algorithms for bicycle routing started to appear. In [34], the authors showed how to effectively search for a best compromise solution in bicycle routing. However, the method only uses two optimization criteria and it does not produce multiple solutions approximating the full Pareto set. In [35], the authors solve a constrained shortest path (CSP) problem in the bicycle routing domain where the second criterion is used as a hard constrain. Recently, in [8], we have explored the use of optimal multi-criteria shortest path algorithms for tri-criteria bicycle routing; however, the proposed algorithm is too slow for interactive route planning. In contrast, our method employs three optimization criteria, it can generate multiple diverse solutions yet is fast enough for practical, interactive use.

III. Model: Multi-Criteria Bicycle Routing Problem

We represent the cycleway network as a weighted directed *cycleway graph* $G = (V, E, g, h, l, f, \overrightarrow{c})$, where V is the set of nodes representing start and end points (e.g., cycleway junctions) of cycleway segments, and $E \subseteq \{(u,v)|(u,v \in V) \wedge (u \neq v)\}$ is the set of edges representing cycleway segments. The cycleway graph is directed to properly model cycleway segments in the map (two edges represent a two-way link; one edge represents a one-way link). We also assume that the cycleway graph is strongly connected. The function $g : V \rightarrow \mathbb{R}^2$ assigns a latitude and a longitude value to each node $v \in V$. An altitude value is assigned to each node by the function $h : V \rightarrow \mathbb{R}$. The horizontal length of each edge $(u,v) \in E$ is given by the function $l : E \rightarrow \mathbb{R}_0^+$. Let $\wp(F)$ denote the powerset of F. For each edge $(u,v) \in E$, the function $f : E \rightarrow \wp(F)$ returns the *features* associated with the edge, which capture relevant properties of the edge as obtained from the input map data (e.g., the surface of the cycleway segment or the road type). The set of all edge features is denoted by F. Note that an edge can have multiple features assigned to it, thus $f((u,v)) \in \wp(F)$ with the number of elements $|f((u,v))| \geq 1$.

The cost of each edge is calculated by a k-dimensional vector of cost functions $\overrightarrow{c} = (c_1, c_2, \ldots, c_k)$. The non-negative cost

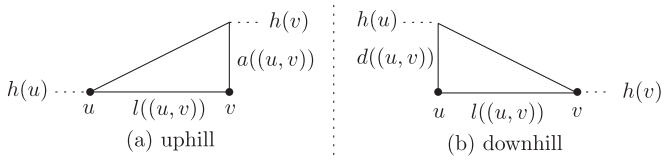


Fig. 1. Positive vertical (a) ascent a and (b) descent d.

value $c_i((u,v))$ of i-th criterion for a given edge $(u,v) \in E$ is computed by the cost function $c_i : E \to \mathbb{R}_0^+$. The *multi-criteria bicycle routing problem* is then defined as a triple $C = (G, o, d)$:

- $G = (V, E, g, h, l, f, \overrightarrow{c})$ is the *cycleway graph*
- $o \in V$ is the route origin
- $d \in V$ is the route destination

A route π, i.e., a finite path $\pi = ((o = u_1, v_1), \ldots, (u_n, v_n = d))$ with a length $|\pi| = n$ from the origin o to the destination d in the cycleway graph G has an additive cost value:

$$\overrightarrow{c}(\pi) = \left(\sum_{j=1}^{|\pi|} c_1(u_j, v_j), \ldots, \sum_{j=1}^{|\pi|} c_k(u_j, v_j) \right).$$

The solution of the multi-criteria bicycle routing problem is a full *Pareto set* of routes $\Pi \subseteq \{\pi | \pi = ((u_1, v_1), \ldots, (u_n, v_n))\}$, i.e., all routes non-dominated by any other route. A route π_p dominates another route π_q, i.e., $\pi_p \prec \pi_q$, iff $c_i(\pi_p) \leq c_i(\pi_q)$ for all $1 \leq i \leq k$ and $c_j(\pi_p) < c_j(\pi_q)$ for at least one j, $1 \leq j \leq k$.

A. Tri-Criteria Bicycle Routing Problem

Based on the studies of real-world cycle route choice behavior [4], [5], we further consider a specific class of multi-criteria bicycle routing problems—a *tri-criteria bicycle routing problem*. Specifically, we consider the travel time criterion c_{time}, the comfort criterion c_{comfort}, and the elevation gain criterion c_{gain} defined in the next subsections. The criteria functions are then instantiated in Section III-B2.

1) Travel Time Criterion: The travel time criterion captures the preference towards routes that can be travelled in a short time. Travel time is a sensitive factor in cyclists' route planning especially for commuting purposes. To model the slowdown caused by obstacle features such as stairs or crossings, we define the slowdown coefficient $r_{\text{slowdown}} : \wp(F) \to \mathbb{R}_0^+$ which returns the slowdown in seconds on the given edge $(u,v) \in E$ with a set of features $f((u,v))$.

Besides, changes in elevation may affect the cyclist's velocity and hence affect travel times. For the case of uphill rides, we define the positive vertical ascent $a : E \to \mathbb{R}_0^+$, cf. Fig. 1.

$$a((u,v)) := \begin{cases} h(v) - h(u) & \text{if } h(v) > h(u) \\ 0 & \text{otherwise.} \end{cases}$$

Analogously, for the case of downhill rides, we define the positive vertical descent $d : E \to \mathbb{R}_0^+$ (cf. Fig. 1) and the positive descent grade $d' : E \to \mathbb{R}_0^+$ for a given edge $(u,v) \in E$ as follows:

$$d((u,v)) := \begin{cases} h(u) - h(v), & \text{if } h(u) > h(v) \\ 0, & \text{otherwise} \end{cases}$$

$$d'((u,v)) := \frac{d((u,v))}{l((u,v))}.$$

To model the speed acceleration caused by vertical descent for a given edge $(u,v) \in E$, we define the downhill speed multiplier $s_d : E \times \mathbb{R}^+ \to \mathbb{R}^+$ as:

$$s_d((u,v), s_{d\max}) := \begin{cases} s_{d\max}, & \text{if } d'((u,v)) > d'_c \\ \frac{(s_{d\max}-1)d'((u,v))}{d'_c} + 1, & \text{otherwise} \end{cases}$$

where $s_{d\max} \in \mathbb{R}^+$ is the maximum downhill speed multiplier, and $d'_c \in \mathbb{R}^+$ is the critical d' value over which a downhill ride would use the multiplier of $s_{d\max}$. This reflects the fact that the speed acceleration is remarkable for the ride on a steep downhill (compared to a mild one), however, it is limited due to safety concerns, bicycle physical limits and air drag.

Considering the integrated effect of edge length, the change in elevation and its associated features, the travel time criterion is defined as:

$$\begin{aligned} c_{\text{time}}((u,v)) &= \frac{\text{distance}}{\text{speed}} + \text{slowdown} \\ &= \frac{l((u,v)) + a_l \cdot a((u,v))}{s \cdot s_d((u,v), s_{d\max}) \cdot r_{\text{time}}((u,v))} \\ &\quad + r_{\text{slowdown}}((u,v)) \end{aligned}$$

where s is the average cruising speed of a cyclist and a_l is the penalty coefficient for uphill rides (we use a modification of the Naismith's rule [36]). The criteria coefficient $r_{\text{time}}((u,v))$ expresses the effect of a set of features $f((u,v))$ assigned to a given edge $(u,v) \in E$. Intuitively, $c_{\text{time}}((u,v))$ can model the travel time of flat rides, uphill rides, and downhill rides with $s_d((u,v), s_{d\max}) = 1$ for uphill and flat scenarios and $a((u,v)) = 0$ for downhill and flat scenarios. Details about the travel time coefficient r_{time} and slowdown coefficient r_{slowdown} are located in Section III-B2.

2) Comfort Criterion: The comfort criterion captures the preference towards comfortable routes with good-quality surfaces and low traffic. The comfort criterion summarizes the effect of road surface and traffic volume along the edge. The surface coefficient $r_{\text{surface}}((u,v))$ penalizes bad road surfaces, obstacles such as steps, and places where the cyclist needs to dismount his/her bicycle, with small values indicating cycling-friendly surfaces. The traffic coefficient $r_{\text{traffic}}((u,v))$ measures traffic volumes by considering the infrastructure for cyclists (e.g., dedicated cycleways), the type of roads, and the junctions, where low-traffic cycleways are assigned a small coefficient value. The maximum coefficient of the two is used here to avoid the cycleway segments that negatively affect the comfort the most. The comfort criteria coefficients are weighted by edge length $l((u,v))$, i.e., 500 m of cobblestones is worse

than 100 m of cobblestones. The criteria function for comfort is then defined as:

$$c_{\text{comfort}}((u,v)) = \max\{r_{\text{surface}}((u,v)), r_{\text{traffic}}((u,v))\} \cdot l((u,v)).$$

3) Elevation Gain Criterion: The elevation gain criterion captures the preference towards flat routes with minimum uphill segments. The criteria function for elevation gain takes into account the average cruising speed s, the positive vertical ascent $a((u,v))$, and the penalty coefficient for uphill rides a_l. The uphill ride over the edge (u,v) is penalized by adding the flat distance $a_l \cdot a((u,v))$. The criteria function for the elevation gain is defined as:

$$c_{\text{gain}}((u,v)) = \frac{\text{distance}}{\text{speed}} = \frac{a_l \cdot a((u,v))}{s}.$$

B. Problem Instantiation From Data

We now describe important implementation details, in particular related to creating instances of the multi-criteria bicycle routing problem (see the definition in Section III) from real-world map data. The instantiation of the routing problem comprises of two steps. In the first step, the cycleway graph is created from map data and its nodes and edges are assigned respective map features. In the second step, the values of the three criteria functions are calculated for each node and edge.

1) Data: OpenStreetMap[1] (OSM) data is used to create the cycleway graph. OSM data is organized into three entities: nodes, ways, and relations, which are associated with various tags (features). Each map feature is denoted by a key and a value in the form of `entity::key::value`, e.g., `way::highway::primary`. Latitude and longitude of each node is mapped to function g, altitude of each node is mapped to h. The following map elements relevant for cyclists are loaded according to the information from OSM tags associated with OSM nodes, ways, and relations. We divide the map features into six categories:

- *Surface*: surface quality in terms of smoothness of the surface and surface material, e.g., asphalt, gravel, or cobblestone.
- *Obstacles*: steps and elevators.
- *Dismount*: places where cyclists need to dismount the bicycle, e.g., pavement or footway crossing.
- *For bicycles*: description of the infrastructure for cyclists, e.g., dedicated cycleway, cycle lane, or shared busway.
- *Motor roads*: category of a road that is also used by cars, e.g., primary, secondary, residential, or living street.
- *Crossings*: crossings, crossroads, or traffic lights on the road.

Geographical locations of all nodes in the OSM data are represented as their latitude and longitude values using the World Geodetic System[2] (version WGS 84), a *geographic* coordinate system type. In order to simplify the complex calculation of the distance between two nodes expressed in the WGS 84 coordinates, we use a *projected* coordinate system. For locations in Prague, the spatial reference system S-JTSK (Ferro)/Krovak[3] is used (the average deviation of the projection is approximately only 20 cm). The horizontal length l of each edge is calculated as Euclidean distance in the projected coordinates. Elevation h for all nodes in the OSM data is acquired using the Shuttle Radar Topography Mission (SRTM) project.[4]

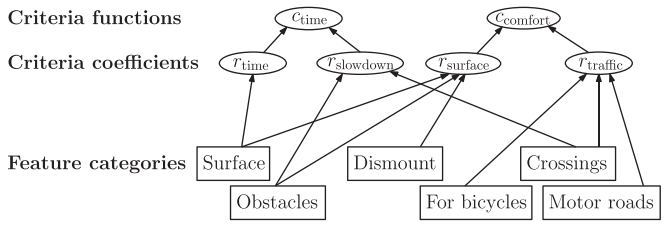


Fig. 2. Calculating criteria values for c_{time} andc_{comfort} from map feature categories. The arrows showwhat are the inputs for the criteria and their criteriacoefficients.

2) OSM Tags Mapping: The three criteria c_{time}, c_{comfort}, and c_{gain} are computed using four criteria coefficients r_{time}, r_{slowdown}, r_{surface}, and r_{traffic} that aggregate the effect of map features $p \in F$. Based on the effect of map features on different criteria, we further map the six categories of features to the optimization criteria, as shown in Fig. 2. To compute the values of criteria functions, we define here the four criteria coefficients—r_{time} and r_{slowdown} for the travel time criterion and r_{surface} and r_{traffic} for the comfort criterion. Their values are assigned using an expert opinion given by a group of chosen cyclists.

The travel time criterion is calculated using the travel time criteria coefficient r_{time} and the slowdown coefficient r_{slowdown}. The travel time criteria coefficient r_{time} is computed using the travel time feature value $r'_{\text{time}} : F \to \mathbb{R}^+$ that represents the influence of a certain feature $p \in F$ on the travel time. The total effect depends on all features associated with an edge that contribute to the travel time criterion. Suppose function $f((u,v))$ returns the features associated with an edge $(u,v) \in E$, then the criteria coefficient r_{time} is given by:

$$r_{\text{time}}((u,v)) = \min\{r'_{\text{time}}(p) | p \in f((u,v))\}.$$

For the travel time criterion, the minimum feature value is used since we are interested in a feature that reduces the cyclist's speed the most. Specific values of r'_{time} for the surface category of tags are shown in Table I.[5]

The slowdown coefficient r_{slowdown} is computed using the slowdown feature value $r'_{\text{slowdown}} : F \to \mathbb{N}_0^+$ caused by each relevant feature $p \in F$. Similarly, the total effect depends on all features associated with the edge $(u,v) \in E$ that may cause a slowdown in seconds, hence the slowdown coefficient $r_{\text{slowdown}} : \wp(F) \to \mathbb{N}_0^+$ is given by:

$$r_{\text{slowdown}}((u,v)) = \max\{r'_{\text{slowdown}}(p) | p \in f((u,v))\}.$$

[1] http://www.openstreetmap.org/
[2] http://spatialreference.org/ref/epsg/wgs-84/
[3] http://spatialreference.org/ref/epsg/2065/
[4] http://www2.jpl.nasa.gov/srtm/
[5] A complete list of feature values is available in the repository: https://github.com/agents4its/cycleplanner/blob/mcspeedups/cycle-planner/src/main/resources/feature_values.csv.

TABLE I
TRAVEL TIME, SLOWDOWN, SURFACE, AND TRAFFIC FEATURE VALUES OVERVIEW (ABBREVIATIONS USED: $r'_{\mathrm{time}}(p) \to r'_{\mathrm{ti}}$, $r'_{\mathrm{slowdown}}(p) \to r'_{\mathrm{sl}}, r'_{\mathrm{surface}}(p) \to r'_{\mathrm{su}}, r'_{\mathrm{traffic}}(p) \to r'_{\mathrm{tr}}$). THE UNIT OF r'_{sl} IS SECONDS; OTHER FEATURE VALUES ARE WITHOUT A UNIT

category:entity:key:value	r'_{ti}	r'_{sl}	r'_{su}	r'_{tr}
surface:way:surface:cobblestone	0.7		5	
surface:way:surface:compacted	0.9		1.5	
surface:way:surface:dirt	0.7		3	
surface:way:surface:grass	0.65		5	
surface:way:surface:gravel	0.5		5	
surface:way:surface:ground	0.6		4	
surface:way:surface:mud	0.4		5	
surface:way:surface:paving_stones	0.75		1.5	
surface:way:surface:sand	0.6		4	
surface:way:surface:setts	0.8		2	
surface:way:surface:unpaved	0.75		4	
surface:way:surface:wood	0.65		4	
obstacles:node:highway:elevator		38		
obstacles:node:highway:steps		8		
obstacles:node:traffic_calming:bump		2		
crossing:node:highway:traffic_signals		15		
crossing:node:highway:stop		8		
crossing:node:crossing:uncontrolled		8		
crossing:node:highway:crossing		8		
for_bicycles:way:highway:cycleway				0.2
for_bicycles:way:cycleway:lane				0.6
for_bicycles:way:cycleway:share_busway				0.7
for_bicycles:way:cycleway:shared_lane				0.8
motor_roads:way:highway:living_street				0.5
motor_roads:way:highway:tertiary				2
motor_roads:way:highway:secondary				6
motor_roads:way:highway:primary				10

We take the maximum value of r'_{slowdown} since we are interested in a feature that slows down the cyclist the most. Specific values of r'_{slowdown} for an example subset of features $p \in F$ are shown in Table I.[5]

The comfort criterion is calculated using the surface criteria coefficient r_{surface} and traffic criteria coefficient r_{traffic} that are computed by surface feature value r'_{surface} and traffic feature value r'_{traffic} respectively. The total effect depends on all comfort related features associated with an edge:

$$r_{\mathrm{surface}}\left((u,v)\right) = \max\left\{r'_{\mathrm{surface}}(p) | p \in \ f\left((u,v)\right)\right\}$$
$$r_{\mathrm{traffic}}\left((u,v)\right) = \max\left\{r'_{\mathrm{traffic}}(p) | p \in \ f\left((u,v)\right)\right\}.$$

Again, we take the maximum values of r'_{surface} and r'_{traffic} because we need to take into account the features that negatively affect the comfort the most. Specific values of r'_{surface} and specific values of r'_{traffic} for an example subset of features from *For bicycles* and *Motor roads* categories are shown in Table I.[5]

IV. ALGORITHM: HEURISTIC-ENABLED MULTI-CRITERIA LABEL-SETTING ALGORITHM

Our newly proposed *heuristic-enabled multi-criteria label-setting* (HMLS) algorithm extends the standard multi-criteria label-setting (MLS) algorithm [9] with several points for inserting speedup heuristic logic. The algorithm uses the following data structures: for each node $u \in V$, $L(u) := (u, (l_1(u), l_2(u), \ldots, l_k(u)), L^P(u))$ represents one of the *labels* at a node u, which is composed of the node u, the cost vector $l(u) = (l_1(u), l_2(u), \ldots, l_k(u))$ indicating the current cost values from the origin to the node u, and the predecessor label $L^P(u)$, which precedes $L(u)$ in an optimal route from an origin. A priority queue Q is used to maintain all labels created during the search. Since each node may be scanned multiple times, we define the bag structure $Bag(u)$ for each node u to maintain the non-dominated labels at u.

The pseudocode of the heuristic-enabled MLS algorithm is given in Algorithm 1. The speedups of the MLS algorithm can be implemented using up to three speedup-specific functions provided by the `Heuristic` interface: `Heuristic.terminationCondition`, `Heuristic.skipEdge`, and `Heuristic.checkDominance`. The logic of the speedup functions is described in Section IV-A. Here we describe the overall heuristic-independent logic which consists of the following steps:

Algorithm 1: Heuristic-enabled MLS algorithm

Input: cycleway graph $G = (V, E, g, h, l, f, \overrightarrow{c})$, origin node o, destination node d
Output: full Pareto set of labels

```
1  Q := empty priority queue
2  Bag(∀v ∈ V) := empty set
3  L(o) := (o, (0, 0, ..., 0), null)
4  Q.insert(L(o))
5  Bag(o).insert(L(o))
6  while Q is not empty do
7      current := Q.pop()
8      u := current.getNode()
9      (l_1(u), l_2(u), ..., l_k(u)) := current.getCost()
10     L^P(u) := current.getPredecessorLabel()
11     if Heuristic.terminationCondition(current) then
           break
12     foreach edge (u, v) do
13         l_i(v) := l_i(u) + c_i(u, v) for i = 1, 2, ..., k
14         next := (v, (l_1(v), l_2(v), ..., l_k(v)), current)
15         if Heuristic.skipEdge(next) then continue
16         if Heuristic.checkDominance(next) then
17             Bag(v).insert(next)
18             Q.insert(next)
19         end
20     end
21 end
22 return Bag(d)
```

Step 1—Initialization: For a k-criteria optimization problem, the algorithm first initializes the priority queue Q and *Bag* for each $v \in V$. Then it initializes the label at the origin to $L(o) := (o, (l_1(o), l_2(o), \ldots, l_k(o)), null)$, where $l_i(o) = 0$ for $i = 1, 2, \ldots, k$. Finally, it inserts the initial label $L(o)$ into the queue Q and the $Bag(o)$.

Step 2—Label Expansion: The algorithm extracts a minimum label $current := (u, (l_1(u), l_2(u), \ldots, l_k(u)), L^P(u))$ from the priority queue Q in a lexicographic order of a cost vector. Given two vectors $(l_1, l_2, \ldots, l_k)$ and $(l'_1, l'_2, \ldots, l'_k)$, $(l_1, l_2, \ldots, l_k) \leq (l'_1, l'_2, \ldots, l'_k)$ if and only if $l_1 < l'_1$ or $((l_1 = l'_1)$ and $(l_2 < l'_2))$ or $\ldots$ or $((l_1 = \cdots = l'_{k-1})$ and $(l_k \leq l'_k))$.

For each outgoing edge (u, v), the algorithm computes the new cost vector $(l_1(v), l_2(v), \ldots, l_k(v))$ by adding the costs of the edge (u, v) to the current cost values $(l_1(u), l_2(u), \ldots, l_k(u))$. Then, it creates a new label *next* using

the node v, the cost vector $(l_1(v), l_2(v), \ldots, l_k(v))$ and the predecessor label *current*.

Function `MLS.skipEdge` (a basic implementation of `Heuristic.skipEdge` on line 15 of Algorithm 1) prevents looping the path by checking the predecessor label in the label data structure, i.e., if previous node $L^P(u).getNode()$ is equal to the node v then the edge (u, v) is skipped [37]. We have also experimented with checking the whole path for cycles. However, this significantly degraded the runtime of the algorithm. Therefore, we only check the predecessor label and larger cycles are eliminated through dominance checks.

Function `MLS.checkDominance` (cf. lines 16–19 of Algorithm 1 and the pseudocode `MLS.checkDominance` in Appendix), by default, controls dominance between the label *next* and all labels inside $Bag(v)$. If *next* is not dominated, the algorithm inserts it into $Bag(v)$ and Q. Also, if some label inside $Bag(v)$ is dominated by the *next* label, it is eliminated from the bag structure $Bag(v)$ and the priority queue Q, i.e., it will not be considered in future search.

Step 3—Pruning Condition: The algorithm exits if the queue Q becomes empty. Otherwise, it continues with *Step 2*.

After the algorithm has finished, the optimal Pareto set of routes Π^* is extracted. Let $|Bag(d)| = |\Pi| = m$. Then, from labels $L_1, \ldots, L_m$ in the destination Pareto set of labels $Bag(d)$, the routes $\pi_1, \ldots, \pi_m$ are extracted using the predecessor labels $L^P(\cdot)$. These routes comprise the set $\Pi^* = \{\pi_1, \pi_2, \ldots, \pi_m\}$ of optimal Pareto routes.

A. Speedups for the HMLS Algorithm

A significant drawback of the standard MLS algorithm is that it is very slow. The main parameter that affects the runtime of the algorithm is the size of the Pareto set. In general, the Pareto set can be exponentially large in the input graph size [14]. Furthermore, the MLS algorithm always explores the whole cycleway graph.

To accelerate the multi-criteria shortest path search, we introduce five speedup heuristics. Two of the heuristics are newly proposed by us: *ratio-based pruning* and *cost-based pruning*, while three are existing heuristics: *ellipse pruning*, *ϵ-dominance* and *buckets*. Implementation-wise, the heuristics are incorporated into the heuristic-enabled MLS algorithm by defining the respective three heuristic-specific functions used in Algorithm 1.

1) Ellipse Pruning: The first speedup heuristic taken from [15] prevents the MLS algorithm from always searching the whole cycleway graph, even for a short origin-destination distance.[6] The heuristic permits visiting only the nodes that are within a predefined ellipse. The focal points of the ellipse correspond to the journey origin o and the destination d. We maintain a constant ratio a/b between semimajor axis a and semiminor axis b. In addition, to improve the ellipse performance for short origin-destination distances, we keep a minimum value $d'_{\min}$ for the length of $d' = |op|$ (the Euclidean distance between the origin o and a peripheral point p on the main axis of the ellipse,

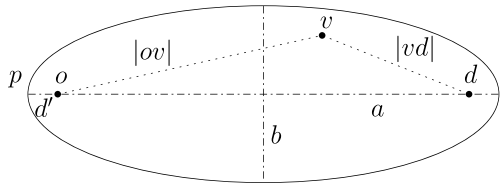


Fig. 3. Geometry of the ellipse pruning condition.

cf. Fig. 3). During the search, in the `Ellipse.skipEdge` function (Algorithm 1, line 15), it is checked whether an edge (u, v) has its target node v inside the ellipse by checking the inequality $|ov| + |vd| \leq 2a$.

2) Ratio-Based Pruning: The ratio-based pruning terminates the search (long) before the priority queue gets empty (which means that the whole search space has been explored). A pruning ratio $\alpha \in \mathbb{R}^+$ is defined and the search is terminated when the chosen criterion cost value $l_i(u)$ in the *current* label exceeds α times the best so far value of the same criterion for a route that has already reached the destination (this is checked in the `RatioPruning.terminationCondition` function, cf. pseudocode of the method in Appendix and Algorithm 1, line 11). In this article, we choose the time criterion $l_1(u)$ since we do not want to consider plans with an excessive duration.

3) Cost-Based Pruning: The third heuristic does not include a label $L(v)$ in the bag of the node v in the situation when $L(v)$ has cost values similar to some label already located in the bag. To be specific, the `CostPruning.checkDominance` function (cf. pseudocode of `CostPruning.checkDominance` in Appendix and Algorithm 1, line 16) returns false when $L(v)$ is dominated by a label inside the bag or its cost-space Euclidean distance to some label inside the bag is lower than $\gamma \in \mathbb{R}^+$. Therefore, the search process is accelerated since fewer labels are inserted into the priority queue and the bag. We do not normalize the criteria values because it would be computationally expensive during the search process (without noticeable benefit).

4) ϵ-Dominance: The fourth strategy we consider is to weaken the dominance checks so as to reduce the number of labels pushed through the network. We use the notion of ϵ-dominance defined in [16]. A newly generated label with a cost vector $\overrightarrow{c}$ is already dominated by the existing label $L(v) \in Bag(v)$ with a cost vector $\overrightarrow{l}$ if $\overrightarrow{l} \prec (1 + \epsilon)\overrightarrow{c}$ and $\epsilon \in \mathbb{R}^+$ (see pseudocode of `EpsilonDominance.checkDominance` in Appendix). Similarly, any existing label satisfying $\overrightarrow{c} \prec (1 + \epsilon)\overrightarrow{l}$ will be removed from the priority queue and the bag of node v.

5) Buckets: The last heuristic defined in [18] discretizes the cost space using buckets for the criteria values. The heuristic is executed in the `Buckets.checkDominance` function (cf. pseudocode `Buckets.checkDominance` in Appendix and Algorithm 1, line 16). A function $bucketValue : \mathbb{R}_0^+ \times \cdots \times \mathbb{R}_0^+ \to \mathbb{N} \times \cdots \times \mathbb{N}$ is used to assign a real cost vector $\overrightarrow{l}$ an integer bucket vector $bucketValue(\overrightarrow{l})$. The function is implemented as $bucketValue(\overrightarrow{l}) := (l_1 - (l_1 \% bucketSize), \ldots, l_k - (l_k \% bucketSize))$ where % is the modulo operation.

[6] Note that in contrast with single-criterion Dijkstra's algorithm, the MLS algorithm does not stop when the destination node is first reached.

TABLE II
GRAPH SIZES FOR THE EXPERIMENTS

Graph	Nodes	Edges	Area
Prague A	4708	13129	Old Town, Vinohrady
Prague B	3343	8915	Strahov, Brevnov
Prague C	5164	14072	Liben, Vysocany
Whole Prague	69544	188868	The whole city of Prague

V. EVALUATION

To evaluate our approach, we consider the real cycleway network of Prague. Prague is a challenging experiment location due to its complex geography and fragmented cycling infrastructure, which raises the importance of proper multi-criteria routing.

A. Experiment Setting

We evaluate our solution in two phases. In the first phase, we use three different cycleway graphs corresponding to three distinct neighborhood-scale areas of the city of Prague (each of the areas covers approximately 10 km^2). We have chosen parts Prague A, Prague B, and Prague C to be different in terms of network density, nature of the cycling network, and terrain topology so as to evaluate the performance of heuristics across a range of conditions. The sizes of the evaluation graphs are described in Table II. The specifics of each evaluation area are the following:

- Prague A: This graph covers a flat city centre area of the Old Town with many narrow cobblestone streets and Vinohrady with the grid layout of streets.
- Prague B: This graph covers a very hilly area of Strahov and Brevnov with many parks.
- Prague C: This graph covers residential areas of Liben and Vysocany further from the city centre. There are many good cyclepaths in this area.

All evaluation cycleway graphs are strongly connected. The size of the evaluation graphs allows us to run the standard MLS algorithm without any speedups—this is necessary for being able to compare the quality of heuristic and optimal solutions.

In the second scale-up phase, we evaluate the best performing heuristics from the first phase on the large city-scale graph of the whole Prague that has 69 544 nodes and 188 868 edges and covers the area of approximately 200 km^2. In this phase, we impose a 15 minutes limit for the search runtime for each origin-destination pair.

For each graph evaluation area, a set of origin-destination pairs generated randomly with a uniform spatial distribution, was used in the evaluation. First, we generated 100 origin-destination pairs for each of graphs Prague A, B, and C. The minimum origin-destination distance is set to 500 m. The longest routes have approximately 4.5 km. We executed the MLS algorithm and the HMLS with all 15 heuristic combinations using the same generated 100 origin-destination pairs for each graph Prague A, B, and C. Therefore, each heuristic combination is evaluated on 300 origin-destination pairs. Then we generated a total of 100 route requests for the whole Prague graph where the origin-destination distance is set to be in the interval from 500 to 10 000 m.

The parameters in the cost functions were set as follows. The average cruising speed is $s = 14$ km/h, the penalty coefficient for uphill is $a_l = 13$ (according to the route choice model developed in the user study [4]), the maximum downhill speed multiplier is $s_{d\max} = 2.5$, and the critical grade value is $d'_c = 0.1$.

Configuration parameters for the heuristics were optimized so as to maximize the ratio between the algorithm runtime and the quality of the solution (see the next section), as measured on the three graphs Prague A, B, and C. Specifically, the following values were chosen: $a/b = 1.25$ for ellipse pruning, $\alpha = 1.6$ for ratio-based pruning, $\gamma = c_1/5$ for cost-based pruning (c_1 corresponds to a criteria value of the first criterion), $\epsilon = 0.05$ for ϵ-dominance, and (15, 2500, 4) for buckets (the triple defines the buckets sizes for the three used criteria).

The multi-criteria route planning algorithm is implemented in JAVA 7. The source code of the used algorithm and speedups is openly available[7] in a repository. The results obtained in the experimental evaluation section are based on running the algorithm on a single core of a 2.4 GHz Intel Xeon E5-2665 processor of a Linux server. The Osmosis 0.41 and osmfilter tools has been used to extract the Prague area from the OSM data dump.

B. Evaluation Metrics

We consider two categories of evaluation metrics: *speed* and *quality*. We use the following to measure the algorithm speed:

- Average runtime in milliseconds for each origin-destination pair together with its standard deviation σ_{runtime}.
- Average speedup in terms of algorithm runtime over the standard, optimal MLS algorithm.

For a multi-criteria optimization problem, solution quality cannot be simply defined in terms of closeness to an optimal solution—instead, we define solution quality in set terms as the closeness to the full Pareto set. To our best knowledge, there is not a universal way to evaluate the quality of multi-criteria solutions. Therefore, we use the following metrics to measure the quality of returned routes in the multi-criteria bicycle routing problem:

- Average distance $d_c(\Pi^*, \Pi)$ of the heuristic Pareto set Π from the optimal Pareto set Π^* in the cost space. Distance $d_c(\pi^*, \pi)$ between two routes π^* and π is measured as the Euclidean distance in the unit three-dimensional space of criteria values normalized to the [0,1] range (min and max values of each criterion are calculated for all plans in the optimal Pareto set Π^* and the heuristic Pareto set Π; the min value is mapped to 0 and the max to 1).

$$d_c(\Pi^*, \Pi) := \frac{1}{|\Pi^*|} \sum_{\pi^* \in \Pi^*} \min_{\pi \in \Pi} d_c(\pi^*, \pi).$$

  Intuitively, $d_c(\pi^*, \pi) = 0.1$ corresponds to a 6% optimality loss in each criterion, assuming the difference to optimum is distributed equally across all three criteria.

[7] https://github.com/agents4its/cycleplanner/tree/mcspeedups

TABLE III
EVALUATION OF THE HEURISTIC PERFORMANCE ON THREE GRAPHS PRAGUE A, B, AND C. PRIMARY SPEED AND QUALITY METRICS ARE MARKED BY BOLD COLUMN HEADINGS. NONDOMINATED HEURISTIC COMBINATIONS WITH RESPECT TO PRIMARY SPEED AND QUALITY METRICS ARE DENOTED BY A BOLD FONT

Heuristic	Speedup	**Runtime [ms]**	σ_{runtime}	$\lvert\Pi\rvert$	$\sigma_{\lvert\Pi\rvert}$	$\mathbf{d_c}$	d_J	$\Pi_{\%}$
MLS	-	898 101	1 103 097	1 647	2 390	-	-	100.00
HMLS+Buckets	585	**1 536**	1 296	40	44	**0.138**	0.323	55.25
HMLS+Cost	129	6 946	5 421	93	63	0.178	0.333	54.66
HMLS+Epsilon	4490	**200**	131	9	7	**0.195**	0.420	54.61
HMLS+Ratio	3	**294 458**	538 357	1 173	1 690	**0.054**	0.076	99.89
HMLS+Ratio+Buckets	1 415	635	818	35	37	0.173	0.347	58.81
HMLS+Ratio+Cost	284	3 161	3 044	80	51	0.211	0.363	58.41
HMLS+Ratio+Epsilon	7 302	123	98	8	5	0.233	0.449	56.32
HMLS+Ellipse	2	**434 170**	956 062	1 637	2 390	**0.005**	0.008	99.98
HMLS+Ellipse+Buckets	1 801	**499**	853	40	44	**0.142**	0.327	55.27
HMLS+Ellipse+Cost	293	3 069	3 897	93	63	0.179	0.334	54.79
HMLS+Ellipse+Epsilon	9 657	**93**	104	9	7	**0.199**	0.423	54.64
HMLS+Ellipse+Ratio	4	**241 675**	598 290	1 171	1 691	**0.056**	0.080	99.86
HMLS+Ellipse+Ratio+Buckets	2 183	**411**	715	35	37	**0.175**	0.349	58.77
HMLS+Ellipse+Ratio+Cost	427	2 103	2 830	80	51	0.211	0.363	58.52
HMLS+Ellipse+Ratio+Epsilon	11 087	**81**	84	8	5	**0.232**	0.448	56.27

- Average distance $d_J(\Pi^*, \Pi)$ of the heuristic Pareto set Π from the optimal Pareto set Π^* in the physical space. Jaccard distance $d_J(\pi^*, \pi)$ [38] is used to measure the dissimilarity between Pareto routes. For routes π^* and π, i.e., sequences of edges, the physical distance is computed by dividing the difference of the sizes of the union and the intersection of the two route sets by the size of their union. Reasonably, the physical distance definition for routes obeys the triangle inequality

$$d_J(\pi^*, \pi) := \frac{|(\pi^* \cup \pi)| - |(\pi^* \cap \pi)|}{|(\pi^* \cup \pi)|}$$

$$d_J(\Pi^*, \Pi) := \frac{1}{|\Pi^*|} \sum_{\pi^* \in \Pi^*} \min_{\pi \in \Pi} d_J(\pi^*, \pi).$$

- Average number of routes $|\Pi|$ in the Pareto set Π together with its standard deviation $\sigma_{|\Pi|}$.
- The percentage of Pareto routes $\Pi_{\%}$ in heuristic Pareto set Π that are equal to routes in the optimal Pareto set Π^*. For instance, if there are 10 routes in the heuristic Pareto set Π and 6 routes are optimal, the percentage $\Pi_{\%} = 60\%$.

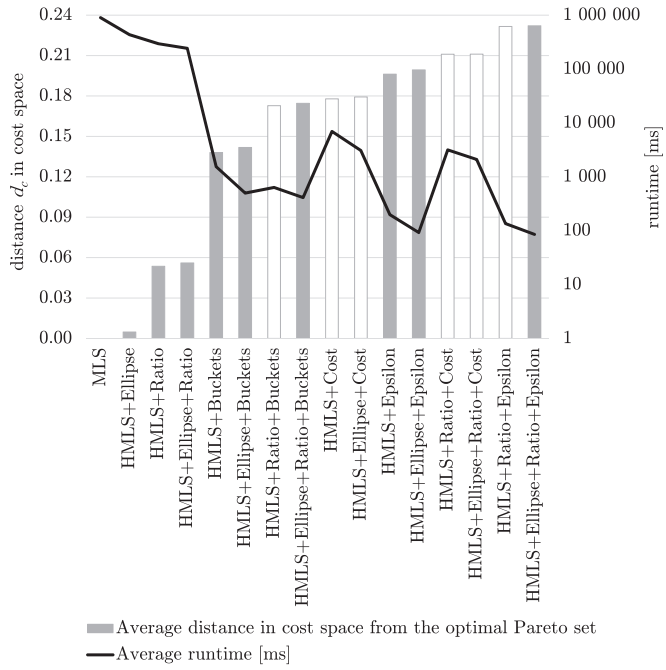


Fig. 4. Speed and quality for the HMLS algorithm and all heuristiccombinations sorted by the quality from the best (MLS on the left-hand side) to the worst. Nondominated heuristic combinations havegray filled-in bars.

C. Results for Graphs Prague A, B, and C

Table III summarizes the evaluation of the HMLS algorithm and its heuristics using the neighborhood-scale graphs Prague A, B, and C. The MLS algorithm is used as a baseline for the evaluation of the proposed heuristics and their combinations. Columns d_c, d_J, and $\Pi_{\%}$ are calculated with respect to the optimal Pareto set Π^* returned by the MLS algorithm. The MLS algorithm returns optimal solutions (1647 routes in the Pareto set on average) at the expense of a prohibitively high runtime.

As anticipated, all heuristic methods are significantly faster than the MLS algorithm. First, we have compared the 15 evaluated methods using the following metrics: the average runtime (from the speed category) and the average distance $d_c(\Pi^*, \Pi)$ in the cost space (from the quality category). From the perspective of these two metrics, there are *nine non-dominated combinations* of heuristics, cf. filled-in bars in Fig. 4 and bold values in Table III. In the following, we only discuss the non-dominated combinations of heuristics.

The *HMLS+Ellipse* heuristic performs best in terms of the quality of the solution. It successfully prunes the search space with $d_c(\Pi^*, \Pi) = 0.005$, i.e., approximately 0.3% optimality loss. The average runtime of this heuristic is around seven minutes. This heuristic is very good for combining with other heuristics, it offers double speedup over the MLS algorithm with a negligible quality loss (99.98% of the routes in the heuristic Pareto set Π are equal to the ones in the optimal Pareto set Π^*).

The *HMLS+Ratio* and *HMLS+Ellipse+Ratio* heuristics offer very good quality with $d_c(\Pi^*, \Pi) = 0.054$ and $d_c(\Pi^*, \Pi) = 0.056$ respectively. Adding the ellipse pruning heuristic cuts the runtime from approximately 5 to 4 minutes while keeping the quality of the solution very similar. The search space is pruned

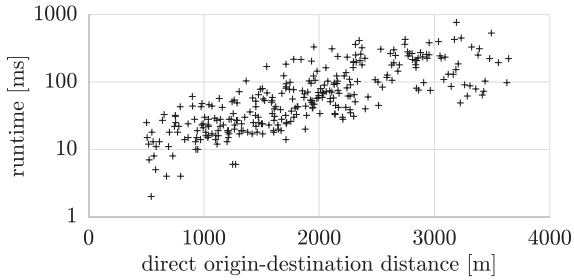


Fig. 5. Runtime of HMLS+Ellipse+Epsilon in milliseconds in dependence on the direct origin–destination distance.

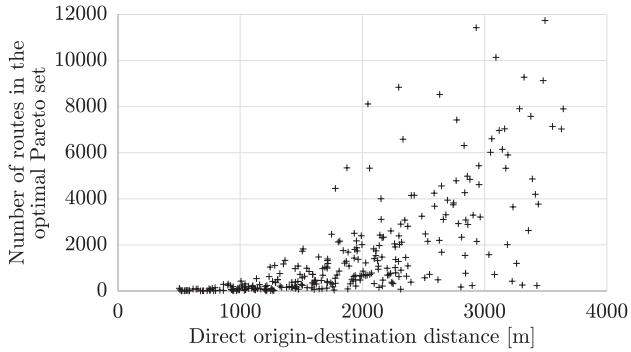


Fig. 6. Number of routes in the optimal Pareto set $|\Pi^*|$ in dependence on the direct origin–destination distance.

geographically by the ellipse pruning and the search is also terminated sooner by the ratio-based pruning method.

With a small decrease of the solution quality to $d_c(\Pi^*, \Pi) = 0.138$, *HMLS+Buckets* heuristic offers a significant additional speedup in average runtime to approximately 1.5 seconds. This makes this heuristic (and also the following ones) usable for real time applications, e.g., a web-based bicycle journey planner. When the ellipse pruning method is combined with the buckets, the average runtime of *HMLS+Ellipse+Buckets* is lowered to approximately 500 ms while keeping very similar quality $d_c(\Pi^*, \Pi) = 0.142$. When the ratio pruning method is added, the average runtime of *HMLS+Ellipse+Ratio+Buckets* is lowered to approximately 400 ms while the quality is decreased to $d_c(\Pi^*, \Pi) = 0.175$.

With an additional worsening of the solution quality to $d_c(\Pi^*, \Pi) = 0.195$, *HMLS+Epsilon* heuristic offers serious speedup in average runtime to approximately 200 milliseconds. When the ellipse pruning method is added to the ϵ-dominance heuristic, the average runtime of *HMLS+Ellipse+Epsilon* is lowered to approximately 93 ms while keeping almost the same quality $d_c(\Pi^*, \Pi) = 0.199$.

The last combination *HMLS+Ellipse+Ratio+Epsilon* performs best in terms of average runtime which is approximately 81 ms, i.e., it has four orders of magnitude speedup over the MLS algorithm. The quality of this combination is reflected by higher $d_c(\Pi^*, \Pi) = 0.232$, still over 56% of the routes in the heuristic Pareto set Π are equal to the ones in the optimal Pareto set Π^*.

To provide a deeper insight in search runtimes, we show in Fig. 5 how the runtime of the *HMLS+Ellipse+Epsilon* heuristic depends on the direct origin-destination distance. The runtime increases with the origin-destination distance since larger search space needs to be explored. This also explains the high σ_{runtime} in Table III. Despite the increase, our scale-up experiments in Section V-D resulted in less than 10 second response times even for 20 times larger cycleway graph covering the whole city of Prague (approximately 200 km^2).

To get further insight in the sizes of generated Pareto sets, we observe that the average number of Pareto routes increases notably with the direct origin-destination distance. In addition, the network structure around the origin and destination also affects the number of Pareto routes. The number of routes in the optimal Pareto set $|\Pi^*|$ in dependence on the direct origin-destination distance is shown in Fig. 6.

The Pareto set can get very large. Therefore, a route selection method needs to be implemented. It is an independent step for which various methods can be used. In fact, we have already proposed a method based on clustering [8]. Regardless of the selection method, the quality of the unfiltered results is essential as high-quality solutions can be selected only if they are included in the unfiltered results set.

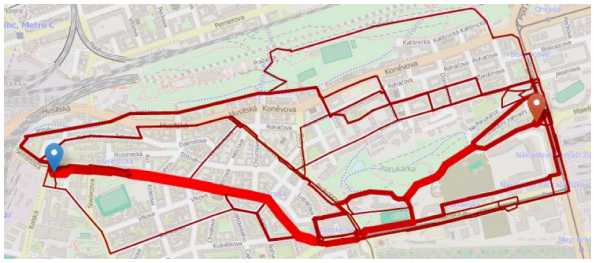
Finally, we have chosen a hilly area in Zizkov to illustrate the Pareto set of routes in the physical space. In Fig. 7, we illustrate the route distribution of the Pareto sets returned by the MLS and HMLS algorithm with three different heuristic combinations. Each subfigure is provided with the name of the heuristic combination, the size of the Pareto set (ranges from 532 to 6 routes) and the algorithm runtime (ranges from 90 seconds to 372 ms). The more routes use a given cycleway network segment, the wider is the depicted line. It can be observed that the heuristics return a Pareto set of routes that very well corresponds to the optimal Pareto set.

To summarize, we have evaluated 15 different combinations of heuristics from which 9 combinations dominated the others in terms of quality and speed. The heuristics offer significant *one to four orders of magnitude speedup* over the MLS algorithm in terms of average runtime. The speedup is achieved by lowering the number of iterations and also the number of dominance checks in each iteration. *HMLS+Ellipse* is the best heuristic in terms of quality of the produced Pareto set while *HMLS+Ellipse+Ratio+Epsilon* is the best heuristic in terms of average runtime. Taking into the account the trade-off between the quality of a solution and the provided speedup, we consider *HMLS+Ellipse+Epsilon* heuristic to have the best ratio between the quality and speed.

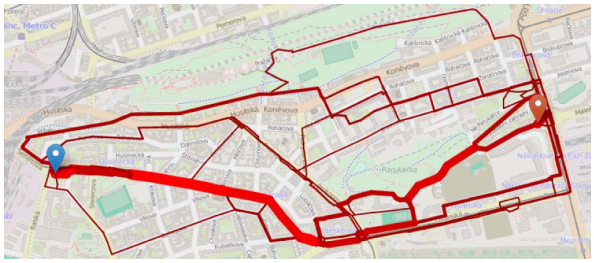
D. Scale-Up Results for the Whole Prague Graph

In order to show the scalability of the proposed heuristics to city-scale routing, we evaluate the heuristics on the graph of the whole Prague city which is approximately 20 times bigger than each of the graphs Prague A, B, and C. The results are summarized in Table IV. We present only a subset of statistics (compared to Table III) since the graph is too large to get an optimal Pareto set by the MLS algorithm. We show the total number of requests finished in the imposed 15-minutes limit, average runtime in milliseconds, and average size of the Pareto set.

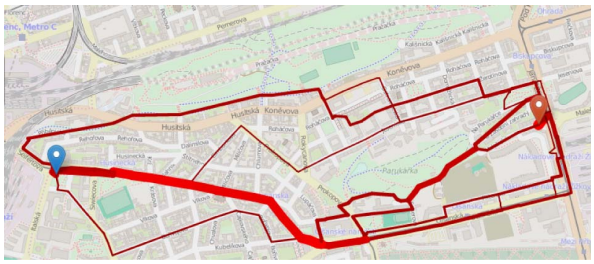
We have evaluated all combinations of heuristics; they offer average runtimes between 5 and 260 seconds. Eight evaluated



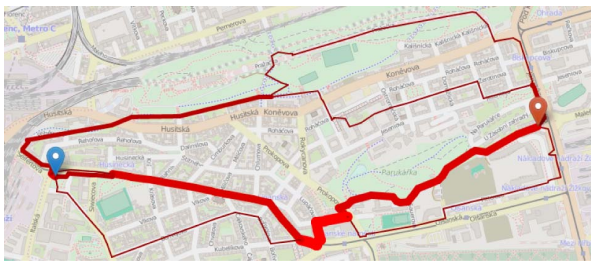
(a)



(b)



(c)



(d)

Fig. 7. Pareto sets for the MLS and HMLS algorithms. (a) MLS, optimal Pareto set with 532 routes, 90 s. (b) HMLS+Ellipse+Ratio, 500 routes, 32 s. (c) HMLS+Ellipse+Buckets, 26 routes, 623 ms. (d) HMLS+Ellipse+Ratio+Epsilon, 6 routes, 372 ms.

TABLE IV
EVALUATION OF THE HEURISTIC PERFORMANCE ON THE WHOLE PRAGUE GRAPH (RUNTIME IS MEASURED IN MILLISECONDS)

Heuristic	15m	Runtime	σ_{runtime}	$\lvert\Pi\rvert$
HMLS+Buckets	0	-	-	-
HMLS+Cost	100	70 734	25 311	72
HMLS+Epsilon	100	15 759	4 859	12
HMLS+Ratio	14	168 302	250 902	402
HMLS+Ratio+Buckets	51	176 810	225 504	126
HMLS+Ratio+Cost	100	49 948	29 421	69
HMLS+Ratio+Epsilon	100	8 847	6 512	11
HMLS+Ellipse	18	260 278	313 214	918
HMLS+Ellipse+Buckets	72	128 096	194 328	244
HMLS+Ellipse+Cost	100	33 474	25 008	72
HMLS+Ellipse+Epsilon	100	**5 306**	5 667	11
HMLS+Ellipse+Ratio	20	112 333	159 589	768
HMLS+Ellipse+Ratio+Buckets	76	130 738	206 001	242
HMLS+Ellipse+Ratio+Cost	100	32 799	25 235	69
HMLS+Ellipse+Ratio+Epsilon	100	**4 834**	5 159	11
NAMOA*+TC	100	106 436	192 884	2 984

heuristic combinations successfully return solutions for all 100 origin-destination pairs in the 15-minutes time limit. We can observe the behavior of the ellipse pruning heuristic that does not significantly decrease the size of the Pareto set yet offers solid speedup consistent with the results on the graphs Prague A, B, and C. The best performing combinations of heuristics are *HMLS+Ellipse+Epsilon* and *HMLS+Ellipse+Ratio+Epsilon* with practically usable average runtime of 5 seconds. This result is consistent with our evaluation of the heuristics on the graphs Prague A, B, C where the two heuristics also performs best in terms of average runtime.

Finally, we compared the HMLS algorithm with heuristic speedups to the optimal NAMOA* algorithm [24] with the Tung & Chew (TC) heuristic [23]. The TC heuristic works as a preprocessing step (for each request) where for each criterion a backward Dijkstra is executed from the destination to all nodes in the graph. The preprocessing calculates the values of a perfect heuristic function, i.e., true cost values from each node to the destination, for the NAMOA* algorithm. Using the whole Prague graph, the preprocessing takes 444 ms on average. The NAMOA* algorithm with the TC heuristic successfully returned solutions for all 100 origin-destination pairs; the total runtime of the algorithm was 106 seconds on average. On the runtime side, the algorithm is better than 6 combinations of heuristics listed in Table IV. However the best performing heuristic combination *HMLS+Ellipse+Epsilon* is 50 times faster than the NAMOA* algorithm with TC heuristic. On the quality side, the algorithm outperformed HMLS algorithm with each heuristic combination since it delivered optimal Pareto sets of routes (2984 on average). Altogether, for the problems where there is not necessary to deliver the whole Pareto set of solutions (e.g., multi-criteria bicycle routing problem) the *HMLS+Ellipse+Epsilon* and *HMLS+Ellipse+Ratio+Epsilon* deliver 50 times better runtime than the NAMOA* algorithm with TC heuristic.

To summarize, we are able to solve multi-criteria bicycle routing problem on the large graph instance with tens of thousands of nodes and edges while achieving practically usable runtimes of 5 seconds.

E. Practical Deployment

As the final steps towards practical deployment of our approach, the presented heuristic multi-criteria routing algorithm is currently being integrated into both web-based frontend and Android routing applications. The applications have been so far using a simplified pseudo-multi-criteria approach described in [30]; in addition to routing, our applications also support turn-by-turn navigation and travelled route tracking. After the integration of the presented novel multi-criteria algorithm has been completed, the cyclists should benefit from higher-quality routes.

VI. CONCLUSION

We have investigated a multi-criteria approach to urban bicycle routing. In contrast to existing work, we have pr ovided a well-grounded formal model of the multi-criteria bicycle routing and have applied a novel heuristic-enabled multi-criteria

shortest path algorithm to find a diverse set of cycling routes. As a result, we have made bicycle routing that properly considers multiple realistic route choice criteria fast enough for practical, interactive use. Our method produces routes closely approximating the full Pareto set in hundreds of milliseconds when routing on a neighborhood scale and in seconds when routing on a city-wide scale. Further speedups are possible through low-level optimization of data structures and the algorithmic logic.

We have achieved these results by employing five heuristic speedup techniques for multi-criteria shortest path search. The speedup heuristics provide a variable trade-off between the search time and the completeness and quality of the suggested routes and they enable fast response times without severely compromising the quality of the results. Although the heuristics are relatively simple, their application in the context of bicycle routing is novel and their effect is very significant. Importantly, the performance of speedup heuristics on real-world instances of multi-criteria bicycle routing problems has not yet been studied. Because real-world speedup performance may differ significantly from the performance on general multi-criteria shortest path problems, our work is important for properly understanding realistic performance trade-offs in developing efficient bicycle routing algorithms. We have evaluated our approach extensively in the challenging conditions of the city of Prague, which features complex geography and fragmented cycling infrastructure. The evaluation has confirmed the usefulness of the multi-criteria approach to bicycle routing.

There are several immediate directions for further improvement of our approach. First, the multi-criteria search produces often large Pareto sets with many similar routes. As a future work, we therefore plan to provide a filtering method (e.g., based on our preliminary work that uses clustering [8]) that would extract several representative routes from a potentially very large set of Pareto routes. Second, we plan to extend the underlying cycleway graph model to consider additional features such as detailed junction models with traffic lights and penalization of turns. Finally, we aim to incorporate real-world GPS tracks we have been collecting into the routing process. This should further improve the quality of route suggestions by making it possible to consider route-choice factors that cannot be directly extracted from the underlying map data.

APPENDIX A
PSEUDOCODES OF METHODS

The appendix contains pseudocode of six methods described in Section IV and IV-A.

Function MLS.checkDominance(label $next$) – Default dominance check

```
global Bag
v := next.getNode()
c⃗ := next.getCost()
foreach label L ∈ Bag(v) do
    l⃗ := L.getCost()
    if l⃗ ≺ c⃗ then return false
    if c⃗ ≺ l⃗ then remove(L)
end
return true
```

Function RatioPruning.terminationCondition(label $current$)

```
global d   // destination
global reachedDestination = false
l⃗ := current.getCost()
u := current.getNode()
upperCost := ∞
if u == d && !reachedDestination then
    upperCost = α · l_1
    reachedDestination := true
end
if l_1 > upperCost then return true
return false
```

Function CostPruning.checkDominance(label $next$)

```
global Bag
v := next.getNode()
c⃗ := next.getCost()
foreach label L ∈ Bag(v) do
    l⃗ := L.getCost()
    if Σ_{i=1}^{k} (c_i − l_i)^2 ≤ γ^2 || l⃗ ≺ c⃗ then return false
    if c⃗ ≺ l⃗ then remove(L)
end
return true
```

Function EpsilonDominance.checkDominance(label $next$)

```
global Bag
v := next.getNode()
c⃗ := next.getCost()
foreach label L ∈ Bag(v) do
    l⃗ := L.getCost()
    if l⃗ ≺ (1 + ε)c⃗ then return false
end
foreach label L ∈ Bag(v) do
    l⃗ := L.getCost()
    if c⃗ ≺ (1 + ε)l⃗ then remove(L)
end
return true
```

Function Buckets.checkDominance(label $next$)

```
global Bag
c⃗ := next.getCost()
foreach label L ∈ Bag do
    l⃗ := L.getCost()
    if bucketValue(l⃗) ≺ bucketValue(c⃗) then    return false
    if bucketValue(c⃗) ≺ bucketValue(l⃗) then    remove(L)
end
return true
```

REFERENCES

[1] D. Herlihy, *Bicycle: The History*. New Haven, CT, USA: Yale Univ. Press, 2004.

[2] C. Dora and M. Phillips, *Transport, Environment and Health*. Copenhagen, Denmark: WHO Regional Office Europe, 2000.

[3] P. L. Jacobsen, "Safety in numbers: More walkers and bicyclists, safer walking and bicycling," *Injury Prevention*, vol. 9, no. 3, pp. 205–209, 2003.

[4] J. Broach, J. Dill, and J. Gliebe, "Where do cyclists ride? A route choice model developed with revealed preference GPS data," *Transp. Res. A, Policy Pract.*, vol. 46, no. 10, pp. 1730–1740, Dec. 2012.

[5] M. Winters, G. Davidson, D. Kao, and K. Teschke, "Motivators and deterrents of bicycling: Comparing influences on decisions to ride," *Transportation*, vol. 38, no. 1, pp. 153–168, Jan. 2011.

[6] V. Filler, "*AUTO*MAT Private Communication*," Why cycle route planners are important for cyclists, 2013.

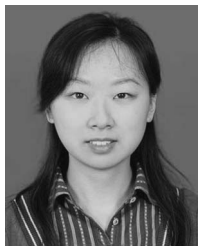
[7] K. Rehrl, S. Bruntsch, and H.-J. Mentz, "Assisting multimodal travelers: Design and prototypical implementation of a personal travel companion," *IEEE Trans. Intell. Transp. Syst.*, vol. 8, no. 1, pp. 31–42, Mar. 2007.
[8] Q. Song, P. Zilecky, M. Jakob, and J. Hrncir, "Exploring pareto routes in multi-criteria urban bicycle routing," in *Proc. 17th Int. Conf. Intell. Transp. Syst.*, Oct. 2014, pp. 1781–1787.
[9] E. Q. V. Martins, "On a multicriteria shortest path problem," *Eur. J. Oper. Res.*, vol. 16, no. 2, pp. 236–245, May 1984.
[10] P. Hansen, "Bicriterion path problems," in *Multiple Criteria Decision Making Theory and Application*, ser. Lecture Notes in Economics and Mathematical Systems. Berlin, Germany: Springer-Verlag, 1980, vol. 177, pp. 109–127.
[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
[12] B. C. Dean, "Continuous-time dynamic shortest path algorithms," M. S. thesis, Dept. Elect. Eng. Comput. Sci., Massachusetts Inst. Technol., Cambridge, MA, USA, 1999.
[13] D. Delling and D. Wagner, "Time-dependent route planning," in *Robust and Online Large-Scale Optimization*. Berlin, Germany: Springer-Verlag, 2009, pp. 207–230.
[14] M. M. Hannemann and K. Weihe, "On the cardinality of the Pareto set in bicriteria shortest path problems," *Ann. Oper. Res.*, vol. 147, no. 1, pp. 269–286, Oct. 2006.
[15] L. Han, H. Wang, and W. Mackey Jr., "Finding shortest paths under time-bandwidth constraints by using elliptical minimal search area," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 1977, pp. 225–233, 2006.
[16] P. Perny and O. Spanjaard, "Near admissible algorithms for multiobjective search," in *Proc. ECAI*, 2008, pp. 490–494.
[17] C. H. Papadimitriou and M. Yannakakis, "On the approximability of trade-offs and optimal access of web sources," in *Proc. 41st Annu. Symp. Found. Comput. Sci.*, 2000, pp. 86–92.
[18] D. Delling, J. Dibbelt, T. Pajor, D. Wagner, and R. F. Werneck, "Computing multimodal journeys in practice," in *Proc. SEA*, 2013, pp. 260–271.
[19] D.-H. Han, Y.-D. Kim, and J.-Y. Lee, "Multiple-criterion shortest path algorithms for global path planning of unmanned combat vehicles," *Comput. Ind. Eng.*, vol. 71, pp. 57–69, May 2014.
[20] B. S. Stewart and C. C. White, "Multiobjective A∗," *J. ACM*, vol. 38, no. 4, pp. 775–814, Oct. 1991.
[21] L. Mandow and J. De La Cruz, "Multiobjective A∗ search with consistent heuristics," *J. ACM*, vol. 57, no. 5, Jun. 2008.
[22] P. Hart, N. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Feb. 1968.
[23] C. T. Tung and K. L. Chew, "A multicriteria pareto-optimal path algorithm," *Eur. J. Oper. Res.*, vol. 62, no. 2, pp. 203–209, 1992.
[24] E. Machuca and L. Mandow, "Multiobjective heuristic search in road maps," *Expert Syst. Appl.*, vol. 39, no. 7, pp. 6435–6445, Jun. 2012.
[25] M. M. Hannemann and M. Schnee, "Finding all attractive train connections by multi-criteria pareto search," in *Proc. ATMOS*, 2004, pp. 246–263.
[26] Y. Disser, M. M. Hannemann, and M. Schnee, "Multi-criteria shortest paths in time-dependent train networks," in *Proc. 7th Int., WEA*, 2008, pp. 347–361.
[27] H. Bast, M. Brodesser, and S. Storandt, "Result diversity for multimodal route planning," in *Proc. 13th Workshop Algorithmic ATMOS*, Sophia Antipolis, France, 2013, pp. 123–136.
[28] H. H. Hochmair and J. Fu, "Web based bicycle trip planning for Broward County, Florida," in *Proc. ESRI User Conf.*, 2009, pp. 1–13.
[29] J. G. Su, M. Winters, M. Nunes, and M. Brauer, "Designing a route planner to facilitate and promote cycling in Metro Vancouver, Canada," *Transp. Res. A, Policy Pract.*, vol. 44, no. 7, pp. 495–505, Aug. 2010.
[30] J. Hrncir, Q. Song, P. Zilecky, M. Nemet, and M. Jakob, "Bicycle route planning with route choice preferences," in *Proc. PAIS*, 2014, pp. 1149–1154.
[31] R. J. Turverey, D. D. Cheng, O. N. Blair, J. T. Roth, G. M. Lamp, and R. Cogill, "Charlottesville bike route planner," in *Proc. SIEDS*, 2010, pp. 68–72.
[32] D. Delling, J. Dibbelt, T. Pajor, D. Wagner, and R. F. Werneck, "Computing and evaluating multimodal journeys," Faculty of Informatics, Karlsruhe Inst. Technol., Karlsruhe, Germany, Tech. Rep. 2012-20, 2012.
[33] D. W. Corne, "The good of the many outweighs the good of the one: Evolutionary multi-objective optimization," in *Proc. IEEE Connections Newslett.*, 2003, pp. 9–13.
[34] G. Sauvanet and E. Neron, "Search for the best compromise solution on multiobjective shortest path problem," *Electron. Notes Discrete Math.*, vol. 36, pp. 615–622, 2010.
[35] S. Storandt, "Route planning for bicycles—exact constrained shortest paths made practical via contraction hierarchy," in *Proc. ICAPS*, L. McCluskey, B. Williams, J. R. Silva, and B. Bonet, Eds. AAAI, 2012, pp. 1–39.
[36] P. Scarf, "Route choice in mountain navigation, Naismith's rule, and the equivalence of distance and climb," *J. Sports Sci.*, vol. 25, no. 6, Apr. 2007.
[37] M. Baum, J. Dibbelt, L. Hubschle-Schneider, T. Pajor, and D. Wagner, "Speed-consumption tradeoff for electric vehicle route planning," in *Proc. ATMOS*, S. Funke, and M. Mihalak, Eds., 2014, pp. 138–151.
[38] M. Levandowsky and D. Winter, "Distance between sets," *Nature*, no. 5323, pp. 34–35, 1971.

Jan Hrnčíř received the master's degree in artificial intelligence from the University of Edinburgh, Edinburgh, U.K., in 2011. He is currently working toward the Ph.D. degree in the Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague, Prague, Czech Republic, in 2011.

He focuses on models and algorithms for sustainable journey planning. In addition, he is interested in standards for scheduled public transport services.

Pavol Žilecký received the master's degree in artificial intelligence from Czech Technical University in Prague, Prague, Czech Republic, in 2015.

Since 2013, he has been with the Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague, as a Software Developer and Researcher. He mainly focuses on applying multicriteria search in the transport domain.



Qing Song received the Ph.D. degree in control theory and control engineering from Shanghai Jiao Tong University, Shanghai, China, in 2012.

In 2013–2014, she was a Postdoctoral Researcher with the Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague, Prague, Czech Republic. Her research interest includes optimization of complex systems.

Michal Jakob received the Ph.D. degree in artificial intelligence and biocybernetics from Czech Technical University in Prague, Praque, Czech Republic, in 2008.

He is a Principal Researcher and the Deputy Head with the Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague. He leads the Agent-based Computing for Intelligent Transport Systems research group. He has published over 50 scientific papers and is a Principal Inventor on two international patents.